EASY DBMS MINI-PROJECT

Personal Expense Tracker

Implementation Plan using Python, Django & MongoDB

Frontend Django Templates + Bootstrap + JavaScript	**Backend** Python + Django	**Database** MongoDB + PyMongo

Project goal
Build a small but complete web application that lets a user record income and expenses, manage categories, search transactions, and view monthly summaries. The application demonstrates real MongoDB operations through a Django-powered frontend and backend.

Why this project?

- Simple, familiar use case with a manageable amount of code.
- Naturally demonstrates MongoDB documents, BSON types, CRUD, validation, indexes, and aggregation.
- Supports a clean demo: add a transaction, filter it, edit/delete it, and show monthly totals.
- Can be expanded later with budgets, CSV export, charts, or optional AI-based spending insights.

Syllabus alignment: based on the supplied CSE494 Intelligent NoSQL Databases syllabus (Units I–VI and practicals).

1. Project scope

Core features (must build)

- Dashboard: total income, total expenses, current-month balance, and a simple monthly chart.
- Transaction CRUD: create, view, edit, and delete income/expense records.
- Categories: create and manage categories such as Food, Travel, Fees, Salary, Shopping, and Bills.
- Search and filters: filter by date range, transaction type, category, and keyword.
- Monthly report: totals by month and category using MongoDB aggregation.
- Input validation: amount must be positive; type must be income or expense; required fields cannot be empty.
- Basic login: use Django authentication for separate user accounts; every transaction must be scoped to its owner.

Optional enhancements (only after the core works)

- Set monthly category budgets and show overspending warnings.
- Export filtered transactions to CSV.
- Use Chart.js for category-wise and month-wise visualizations.
- Optional AI extension: simple spending-pattern summaries; not required for the core DBMS project.

2. Technology stack

Layer	Choice	Reason
Frontend	Django templates, HTML, CSS, Bootstrap, JavaScript	Easy to build; responsive forms and tables without a separate frontend server.
Backend	Python + Django	URL routing, views, authentication, form validation, and business rules.
Database driver	PyMongo	Direct control over MongoDB collections, CRUD, indexes, and aggregation.
Database	MongoDB Community Edition	Local document database for development and demonstration.
Tools	MongoDB Compass + mongosh + VS Code	Inspect documents and run/test MongoDB queries.
Charts	Chart.js (optional)	Display monthly income and expenses.

Important integration note

Django's built-in ORM is designed for relational databases. For this plan, use Django for the web application and PyMongo directly for MongoDB access. Do not attempt to use Django ORM models as if MongoDB were a standard relational backend. Use Django's authentication framework separately and keep transaction data in MongoDB.

3. MongoDB database design (highest priority)

Database name: **expense_tracker_db**. Create three main collections: **users**, **categories**, and **transactions**. Keep user ownership explicit so one user cannot see another user's data.

Collection	Fields	Purpose / notes
users	_id: ObjectId; username: string; email: string; password_hash: string; created_at: Date	Optional app-owned user documents. If using Django auth, Django users normally live in Django's configured auth database instead; avoid duplicate user stores unless necessary.
categories	_id: ObjectId; user_id: string/ObjectId; name: string; kind: string; created_at: Date	Categories belong to a user. kind can be income, expense, or both.
transactions	_id: ObjectId; user_id: string/ObjectId; category_id: ObjectId; type: string; amount: Decimal128; note: string; transaction_date: Date; created_at: Date; updated_at: Date	Each document records one income or expense. Store references to category and owner.

Recommended modelling decisions

- **Referencing:** transaction documents store category_id instead of duplicating the full category document. This demonstrates a one-to-many relationship: one category can be used by many transactions.
- **Embedding:** if you add optional receipt metadata, store a small receipt object inside the transaction. This demonstrates an embedded document without creating an unnecessary collection.
- **Many-to-many:** not required for the minimum viable project. If the course assessor expects a direct demonstration, add transaction tags as an array of tag IDs or a small separate tag model and explain the trade-offs.
- **BSON types:** use ObjectId for IDs, Decimal128 for currency, Date for dates, strings for labels/notes, and arrays/embedded documents for optional metadata.
- **Ownership:** every category and transaction query must include the logged-in user's ID.

Relationship overview

```
Django-authenticated user
        |
        +----< categories
        |       _id
        |       user_id
        |       name
        |
        +----< transactions
                _id
                user_id
                category_id  ------> categories._id
                type
                amount
                transaction_date
```

Example transaction document

```
{
  "_id": ObjectId("..."),
  "user_id": "django-user-123",
  "category_id": ObjectId("..."),
  "type": "expense",
  "amount": Decimal128("249.50"),
  "note": "Lunch",
  "transaction_date": ISODate("2026-10-19T00:00:00Z"),
  "created_at": ISODate("2026-10-19T10:00:00Z"),
  "updated_at": ISODate("2026-10-19T10:00:00Z")
}
```

Note: user_id representation depends on the chosen Django authentication integration. Pick one stable representation and use it consistently in all documents.

4. Database operations you must demonstrate

CRUD operations

Operation	Example in the application
Create	Add an income or expense transaction using a Django form and PyMongo insert_one().
Read	Display transactions using find(), sort(), limit(), and pagination.
Update	Edit a note, amount, category, or date using update_one().
Delete	Delete a transaction after checking ownership and asking for confirmation using delete_one().

Useful mongosh examples

```
use expense_tracker_db

// Find recent expenses
db.transactions.find({ type: "expense" })
  .sort({ transaction_date: -1 })
  .limit(10)

// Update a note (replace the ID with a real ObjectId)
db.transactions.updateOne(
  { _id: ObjectId("REPLACE_WITH_REAL_ID") },
  { $set: { note: "Groceries" } }
)

// Count expenses
db.transactions.countDocuments({ type: "expense" })
```

Indexes to create

```
// User dashboard: transactions by date
db.transactions.createIndex({ user_id: 1, transaction_date: -1 })

// Filter by type and date for a user
db.transactions.createIndex({
  user_id: 1,
  type: 1,
  transaction_date: -1
})

// Category lookup
db.transactions.createIndex({ user_id: 1, category_id: 1 })
```

Explain index performance

```
db.transactions.find({
  user_id: "django-user-123",
  type: "expense"
}).explain("executionStats")
```

Record the execution plan, totalDocsExamined, totalKeysExamined, and executionTimeMillis before and after creating the relevant index. Use a reasonably sized sample dataset and report measured results; do not claim a performance improvement unless your experiment shows it.

5. Aggregation pipelines and validation

Monthly totals

```
db.transactions.aggregate([
  { $match: {
      user_id: "django-user-123",
      transaction_date: {
        $gte: ISODate("2026-10-01T00:00:00Z"),
        $lt: ISODate("2026-11-01T00:00:00Z")
      }
  }},
  { $group: {
      _id: "$type",
      totalAmount: { $sum: "$amount" },
      transactionCount: { $sum: 1 }
  }},
  { $sort: { totalAmount: -1 } }
])
```

Category-wise expense report

```
db.transactions.aggregate([
  { $match: {
      user_id: "django-user-123",
      type: "expense"
  }},
  { $group: {
      _id: "$category_id",
      totalSpent: { $sum: "$amount" },
      count: { $sum: 1 }
  }},
  { $sort: { totalSpent: -1 } }
])
```

For the dashboard, run separate aggregation pipelines for income, expenses, and category totals. Use $lookup only when you need category names from the categories collection. MongoDB Decimal128 arithmetic should be handled carefully in Python; convert Decimal128 values to Python Decimal for display/calculation, and avoid binary floating-point for currency.

Validation rules

- Application-level validation in Django: amount > 0, type is exactly income or expense, note length is limited, date is valid, and category belongs to the logged-in user.
- Database-level validation: configure MongoDB JSON Schema validation for required fields and BSON types; use enum for transaction type and minimum 0 for amount where supported.
- Use server-side validation even when frontend JavaScript validation exists. Frontend validation is for usability, not security.
- Handle missing documents, invalid ObjectId values, duplicate category names (if prohibited), and database connection failures gracefully.

6. UI pages

Page	What it contains
Login / logout	Django authentication forms and access protection.
Dashboard	Income total, expense total, balance, recent transactions, monthly chart.
Transactions	List, search, filter, sort, paginate, edit, delete.
Add / edit transaction	Type, amount, category, date, note.
Categories	Add, list, edit, delete categories with usage checks.
Reports	Monthly totals, category breakdown, selected date range.

7. Suggested folder structure

```
expense_tracker/
|-- manage.py
|-- requirements.txt
|-- .env
|-- config/
|   |-- settings.py
|   |-- urls.py
|   `-- wsgi.py
|-- tracker/
|   |-- urls.py
|   |-- views.py
|   |-- forms.py
|   |-- validators.py
|   |-- services.py
|   `-- tests.py
|-- database/
|   |-- connection.py
|   |-- collections.py
|   |-- indexes.py
|   |-- validators.js
|   `-- seed_data.py
|-- templates/
|   |-- base.html
|   |-- dashboard.html
|   |-- transactions.html
|   |-- transaction_form.html
|   |-- categories.html
|   `-- reports.html
`-- static/
    |-- css/
    |-- js/
    `-- images/
```

Keep MongoDB access inside reusable service functions rather than scattering raw queries across every view. Store the connection URI in .env, add .env to .gitignore, and never send database credentials to the browser.

8. Step-by-step build schedule

Phase	Work	Done when...
1 — Setup	Install Python, Django, MongoDB; create venv; install PyMongo; verify connection.	A test document can be inserted and read.
2 — Data model	Create collections, seed sample data, define BSON types and validation.	Documents follow the agreed structure.
3 — Core CRUD	Build categories and transaction create/read/update/delete.	All CRUD actions work from the website.
4 — User access	Add Django login and scope all records by user ID.	Users cannot view or modify other users' data.
5 — Reports	Build dashboard totals and aggregation pipelines.	Displayed totals match known test records.
6 — Indexes	Create indexes and compare explain execution statistics.	A short evidence table documents results.
7 — Finish	Test edge cases, screenshots, report, viva demo.	All checklist items pass.

Estimated duration: 5–8 focused days for a basic version, or 7–10 days including reports, validation, indexing experiments, documentation, and polish. This is a planning estimate, not a guarantee.

9. Mapping to your CSE494 syllabus

Syllabus unit / practical	Project evidence to show
Unit I: NoSQL vs SQL, CAP theorem, data models, architecture	Short report section comparing relational and document databases; explain where the project uses MongoDB.
Install MongoDB, Compass, mongosh, Atlas overview	Screenshot of local server, Compass database, and mongosh commands. Atlas is an optional deployment overview.
Unit II: databases, collections, documents, BSON, CRUD	Show collections and sample BSON documents; demonstrate create/read/update/delete from UI and mongosh.
Unit II: indexes and query optimization	Create the compound indexes and compare explain execution statistics.
Unit III: embedding/referencing, relationship patterns, validation	Explain category references and embedded receipt metadata; show JSON Schema validator. Document the one-to-many relationship.
Unit IV: single, compound, text indexes; covered queries and explain()	Implement appropriate indexes, test text search if added, and include execution plans. Add a covered-query experiment if feasible.
Unit IV: aggregation framework	Show $match, $group, $sort and optionally $lookup in monthly and category reports.
Unit V: Python + PyMongo	Show connection module and CRUD/aggregation service functions.
Unit V: AI-based predictive analytics / AI query optimization	Optional extension only if time allows: basic spending trend prediction or a written evaluation. Do not claim AI optimization unless actually implemented and tested.
Unit VI: mini-project and production best practices	Working demo, validation, authentication, error handling, .env secrets, test cases, project report.
Practical: Node.js + Mongoose integration	Separate small optional CRUD demonstration if your instructor expects every listed practical, because the main app uses Python/PyMongo.

This mapping is based on the syllabus file supplied with the request. Some advanced topics, especially AI analytics and Node.js/Mongoose, are syllabus topics or practicals but are not necessary for the smallest working version of this Python/Django application.

10. Final submission checklist

- Working local application with login, dashboard, categories, and transaction CRUD.
- MongoDB database with documented collections and realistic sample data.
- Screenshots showing the UI and corresponding documents in MongoDB Compass.
- CRUD queries, BSON examples, validation rules, and index definitions.
- At least two aggregation examples and a query-performance experiment using explain().
- Test cases for invalid amounts, missing categories, invalid IDs, and user data isolation.
- Architecture diagram, data model explanation, setup instructions, limitations, and future scope.
- Short demo flow: log in → add expense → edit it → filter transactions → show dashboard totals → inspect MongoDB → explain an index and aggregation.

Recommended scope discipline

First finish login, categories, and transaction CRUD. Then add aggregation reports, validation, and indexes. Only after these work should you add charts, CSV export, budgets, AI features, or the optional Node.js practical. A smaller project that works correctly is stronger than a larger project with incomplete database operations.